

Rushang Bagada
U24CS076

(1). Write a C++ program to create a simple system for calculating the area of different shapes (Circle and Rectangle) that demonstrates: • Pointer to Function – to call functions like inputData() or displayResult(). • Pointer to Data Member – to access or modify the dimensions of the shapes. • Pointer to Object – to access members of objects dynamically. • this Pointer – to display the current object's details. • Virtual Function – to define a common area() function in the base class. • Runtime Polymorphism – to call the derived class area() functions through a base class pointer.

```
// Q1: Shape Area Calculation System
// Demonstrates: Pointer to Function, Pointer to Data Member, Pointer to
// Object,
// this Pointer, Virtual Function, Runtime Polymorphism

#include <bits/stdc++.h>
using namespace std;

class Shape {
protected:
    string name;
public:
    Shape(string n = "Shape") : name(n) {}

    // Virtual function for polymorphism
    virtual double area() = 0;

    // Function to input data
    virtual void inputData() = 0;

    // Function to display result
    void displayResult() {
        cout << "Shape: " << name << endl;
        cout << "Area: " << area() << endl;
    }

    // Using this pointer
    void showDetails() {
        cout << "Current object address: " << this << endl;
    }
}
```

```

        cout << "Object name: " << this->name << endl;
    }

    virtual ~Shape() {}
};

class Circle : public Shape {
public:
    double radius;

    Circle() : Shape("Circle"), radius(0) {}

    void inputData() override {
        cout << "Enter radius of circle: ";
        cin >> radius;
    }

    double area() override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
public:
    double length, width;

    Rectangle() : Shape("Rectangle"), length(0), width(0) {}

    void inputData() override {
        cout << "Enter length and width of rectangle: ";
        cin >> length >> width;
    }

    double area() override {
        return length * width;
    }
};

int main() {
    cout << "=== Shape Area Calculation System ===\n\n";

```

```

// Pointer to Object - dynamically creating objects
Circle* c = new Circle();
Rectangle* r = new Rectangle();

cout << "--- Circle Input ---\n";
c->inputData();

cout << "\n--- Rectangle Input ---\n";
r->inputData();

cout << "\n--- Results ---\n";

// Pointer to Function - storing function address
void (Shape::*funcPtr)() = &Shape::displayResult;
(c->*funcPtr)(); // Call through pointer to function
cout << endl;
(r->*funcPtr)();

cout << "\n--- Using this Pointer ---\n";
c->showDetails();
cout << endl;
r->showDetails();

// Pointer to Data Member - accessing data members through pointers
cout << "\n--- Pointer to Data Member ---\n";
double Circle::*radiusPtr = &Circle::radius;
double Rectangle::*lengthPtr = &Rectangle::length;
double Rectangle::*widthPtr = &Rectangle::width;

cout << "Circle radius (via pointer): " << c->*radiusPtr << endl;
cout << "Rectangle length (via pointer): " << r->*lengthPtr << endl;
cout << "Rectangle width (via pointer): " << r->*widthPtr << endl;

// Modifying through pointer to data member
c->*radiusPtr = 10.0;
r->*lengthPtr = 15.0;
r->*widthPtr = 20.0;

cout << "\n--- After Modification ---\n";

```

```

    cout << "New Circle radius: " << c->*radiusPtr << endl;
    cout << "New Rectangle dimensions: " << r->*lengthPtr << " x " <<
r->*widthPtr << endl;

    // Runtime Polymorphism - calling through base class pointer
    cout << "\n=== Runtime Polymorphism Demo ===\n";
    Shape* shapes[2];
    shapes[0] = c;
    shapes[1] = r;

    for (int i = 0; i < 2; i++) {
        cout << "\nShape " << i + 1 << ":\n";
        cout << "Area: " << shapes[i]->area() << endl; // Calls derived
class version
    }

    // Cleanup
    delete c;
    delete r;

    return 0;
}

```

(2) Write a C++ program to manage employee details using classes and dynamic memory allocation. Create a class Employee with data members name, age, and salary. Include functions to: Input and display employee details (use the this pointer to show the current object); and update salary by a given percentage. In main(), dynamically create employee objects using pointers, call member functions through them, and demonstrate the use of the this pointer.

```

// Q2: Employee Management System
// Demonstrates: Dynamic memory allocation, this pointer, pointer to
objects

#include <bits/stdc++.h>
using namespace std;

class Employee {
private:

```

```

    string name;
    int age;
    double salary;

public:
    // Constructor
    Employee(string n = "", int a = 0, double s = 0.0) : name(n), age(a),
salary(s) {}

    // Input employee details
    void inputDetails() {
        cout << "Enter name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter age: ";
        cin >> age;
        cout << "Enter salary: ";
        cin >> salary;
    }

    // Display employee details using this pointer
    void displayDetails() {
        cout << "\n=== Employee Details ===\n";
        cout << "Current object address: " << this << endl;
        cout << "Name: " << this->name << endl;
        cout << "Age: " << this->age << endl;
        cout << "Salary: $" << this->salary << endl;
    }

    // Update salary by percentage using this pointer
    void updateSalary(double percentage) {
        this->salary += this->salary * (percentage / 100.0);
        cout << "\nSalary updated by " << percentage << "%\n";
        cout << "New salary: $" << this->salary << endl;
    }

    // Compare two employees using this pointer
    bool earnMoreThan(Employee* other) {
        return this->salary > other->salary;
    }

```

```

// Return this pointer
Employee* getThis() {
    return this;
}

// Demonstrate this pointer explicitly
void showThisPointer() {
    cout << "\nDemonstrating 'this' pointer:\n";
    cout << "Address of current object: " << this << endl;
    cout << "Value accessed via this->name: " << this->name << endl;
    cout << "Address of name member: " << &(this->name) << endl;
}
};

int main() {
    cout << "=== Employee Management System ===\n\n";

    int numEmployees;
    cout << "Enter number of employees: ";
    cin >> numEmployees;

    // Dynamically create array of Employee pointers
    Employee** employees = new Employee*[numEmployees];

    // Input details for each employee
    for (int i = 0; i < numEmployees; i++) {
        cout << "\n--- Employee " << i + 1 << " ---\n";
        employees[i] = new Employee(); // Dynamically allocate each
object
        employees[i]->inputDetails();
    }

    // Display all employee details
    cout << "\n\n=== All Employee Details ===\n";
    for (int i = 0; i < numEmployees; i++) {
        cout << "\nEmployee " << i + 1 << ":";
        employees[i]->displayDetails();
    }
}

```

```

// Update salary for first employee
if (numEmployees > 0) {
    cout << "\n\n=== Updating Salary ===\n";
    double percentage;
    cout << "Enter percentage to increase salary for Employee 1: ";
    cin >> percentage;
    employees[0]->updateSalary(percentage);
    employees[0]->displayDetails();
}

// Demonstrate this pointer
if (numEmployees > 0) {
    cout << "\n\n=== this Pointer Demonstration ===\n";
    employees[0]->showThisPointer();

    // Using getThis() to get the object pointer
    Employee* ptr = employees[0]->getThis();
    cout << "\nPointer returned by getThis(): " << ptr << endl;
    cout << "Original object address: " << employees[0] << endl;
    cout << "Both addresses match: " << (ptr == employees[0] ? "Yes" :
"No") << endl;
}

// Compare employees
if (numEmployees > 1) {
    cout << "\n\n=== Comparing Employees ===\n";
    if (employees[0]->earnMoreThan(employees[1])) {
        cout << "Employee 1 earns more than Employee 2\n";
    } else {
        cout << "Employee 2 earns more than or equal to Employee 1\n";
    }
}

// Demonstrate pointer to member function
cout << "\n\n=== Pointer to Member Function ===\n";
void (Employee::*displayFunc) () = &Employee::displayDetails;
if (numEmployees > 0) {
    cout << "Calling displayDetails() through pointer to member
function:\n";
    (employees[0]->*displayFunc) ();
}

```

```

    }

    // Cleanup - delete all dynamically allocated memory
    for (int i = 0; i < numEmployees; i++) {
        delete employees[i];
    }
    delete[] employees;

    cout << "\n\nMemory cleaned up successfully!\n";

    return 0;
}

```

(3) Write a C++ program to calculate vehicle service cost using virtual functions. Create a base class Vehicle with a virtual function serviceCost() and a normal function displayType(). Derive two classes: Car – cost = baseCost + (engineCC * 0.5) and Bike – cost = baseCost + (engineCC * 0.2). In main(), use base class pointers to call the functions for Car and Bike objects and demonstrate runtime polymorphism.

```

// Q3: Vehicle Service Cost Calculator
// Demonstrates: Virtual functions, Runtime polymorphism

#include <bits/stdc++.h>
using namespace std;

class Vehicle {
protected:
    string name;
    double baseCost;
    int engineCC;

public:
    Vehicle(string n = "Vehicle", double base = 0, int cc = 0)
        : name(n), baseCost(base), engineCC(cc) {}

    // Virtual function for calculating service cost
    virtual double serviceCost() {

```



```

        return baseCost;
    }

    // Normal function
    void displayType() {
        cout << "Vehicle Type: " << name << endl;
    }

    void inputDetails() {
        cout << "Enter base service cost: $";
        cin >> baseCost;
        cout << "Enter engine CC: ";
        cin >> engineCC;
    }

    virtual void displayDetails() {
        cout << "\n=== " << name << " Details ===\n";
        cout << "Base Cost: $" << baseCost << endl;
        cout << "Engine CC: " << engineCC << endl;
        cout << "Total Service Cost: $" << serviceCost() << endl;
    }

    virtual ~Vehicle() {}
};

class Car : public Vehicle {
public:
    Car() : Vehicle("Car", 0, 0) {}

    // Override serviceCost: cost = baseCost + (engineCC * 0.5)
    double serviceCost() override {
        return baseCost + (engineCC * 0.5);
    }

    void displayDetails() override {
        cout << "\n=== Car Service Details ===\n";
        displayType();
        cout << "Base Cost: $" << baseCost << endl;
        cout << "Engine CC: " << engineCC << endl;
    }
};

```

```

        cout << "Additional Cost (CC * 0.5): $" << (engineCC * 0.5) <<
endl;

        cout << "Total Service Cost: $" << serviceCost() << endl;
    }
};

class Bike : public Vehicle {
public:
    Bike() : Vehicle("Bike", 0, 0) {}

    // Override serviceCost: cost = baseCost + (engineCC * 0.2)
    double serviceCost() override {
        return baseCost + (engineCC * 0.2);
    }

    void displayDetails() override {
        cout << "\n=== Bike Service Details ===\n";
        displayType();
        cout << "Base Cost: $" << baseCost << endl;
        cout << "Engine CC: " << engineCC << endl;
        cout << "Additional Cost (CC * 0.2): $" << (engineCC * 0.2) <<
endl;

        cout << "Total Service Cost: $" << serviceCost() << endl;
    }
};

int main() {
    cout << "=== Vehicle Service Cost Calculator ===\n\n";

    // Create objects
    Car car;
    Bike bike;

    // Input details
    cout << "--- Enter Car Details ---\n";
    car.inputDetails();

    cout << "\n--- Enter Bike Details ---\n";
    bike.inputDetails();
}

```

```

// Display using object's own method
car.displayDetails();
bike.displayDetails();

// Runtime Polymorphism - using base class pointers
cout << "\n\n=== Runtime Polymorphism Demo ===\n";
cout << "Calling through base class pointer:\n";

Vehicle* vehicles[2];
vehicles[0] = &car;
vehicles[1] = &bike;

for (int i = 0; i < 2; i++) {
    cout << "\nVehicle " << i + 1 << ":\n";
    vehicles[i]->displayType();
    cout << "Service Cost: $" << vehicles[i]->serviceCost() << endl;
}

// Demonstrate virtual function behavior
cout << "\n\n=== Virtual Function Demonstration ===\n";
Vehicle* vptr;

vptr = &car;
cout << "Base pointer pointing to Car:\n";
cout << "Service Cost: $" << vptr->serviceCost() << " (Car's version
called)\n";

vptr = &bike;
cout << "\nBase pointer pointing to Bike:\n";
cout << "Service Cost: $" << vptr->serviceCost() << " (Bike's version
called)\n";

// Compare costs
cout << "\n\n=== Cost Comparison ===\n";
if (car.serviceCost() > bike.serviceCost()) {
    cout << "Car service is more expensive by $"
        << (car.serviceCost() - bike.serviceCost()) << endl;
} else {
    cout << "Bike service is more expensive by $"
        << (bike.serviceCost() - car.serviceCost()) << endl;
}

```

```

    }

    return 0;
}

```

(4) Write a C++ program to develop a simple Banking System - that demonstrates: • Pointer to Function – to call operations such as deposit(), withdraw(), and displayBalance(). • Pointer to Data Member – to update the balance of an account. • Pointer to Object – to create and access multiple Account objects dynamically. • this Pointer – to identify the current active account. • Virtual Function and Runtime Polymorphism – to implement a base class Account with a • virtual calculateInterest() function, overridden in derived classes SavingsAccount and • CurrentAccount. Requirements: 1. Create a base class Account with members: name, accNo, and balance. 2. Create derived classes SavingsAccount and CurrentAccount. 3. Use runtime polymorphism to call the calculateInterest() function for each type of account through a base class pointer. 4. Demonstrate all pointer types as listed above.

```

// Q4: Banking System
// Demonstrates: Pointer to Function, Pointer to Data Member, Pointer to
// Object,
//               this Pointer, Virtual Function, Runtime Polymorphism

#include <bits/stdc++.h>
using namespace std;

class Account {
protected:
    string name;
    long accNo;
    double balance;

public:
    Account(string n = "", long acc = 0, double bal = 0.0)
        : name(n), accNo(acc), balance(bal) {}

    // Virtual function for calculating interest
    virtual double calculateInterest() = 0;
}

```

```

// Function to deposit money
void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        cout << "Deposited: $" << amount << endl;
        cout << "New Balance: $" << balance << endl;
    } else {
        cout << "Invalid deposit amount!\n";
    }
}

// Function to withdraw money
void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        cout << "Withdrawn: $" << amount << endl;
        cout << "New Balance: $" << balance << endl;
    } else {
        cout << "Invalid withdrawal amount or insufficient
balance!\n";
    }
}

// Function to display balance
void displayBalance() {
    cout << "Current Balance: $" << balance << endl;
}

// Display account details using this pointer
void displayDetails() {
    cout << "\n=== Account Details ===\n";
    cout << "Current object address: " << this << endl;
    cout << "Account Holder: " << this->name << endl;
    cout << "Account Number: " << this->accNo << endl;
    cout << "Balance: $" << this->balance << endl;
}

// Return this pointer
Account* getThis() {
    return this;
}

```

```

    }

    // Get balance (for pointer to data member demo)
    double getBalance() { return balance; }
    void setBalance(double bal) { balance = bal; }

    // Get pointer to balance member (needed for pointer to data member
demo)
    static double Account::* getBalancePointer() {
        return &Account::balance;
    }

    virtual ~Account() {}
};

class SavingsAccount : public Account {
private:
    double interestRate; // Annual interest rate in percentage

public:
    SavingsAccount(string n = "", long acc = 0, double bal = 0.0, double
rate = 4.0)
        : Account(n, acc, bal), interestRate(rate) {}

    // Override calculateInterest
    double calculateInterest() override {
        double interest = balance * (interestRate / 100.0);
        cout << "\n--- Savings Account Interest ---\n";
        cout << "Interest Rate: " << interestRate << "%\n";
        cout << "Interest Amount: $" << interest << endl;
        return interest;
    }

    void applyInterest() {
        double interest = calculateInterest();
        balance += interest;
        cout << "Interest applied. New balance: $" << balance << endl;
    }

    void displayDetails() {

```

```

        Account::displayDetails();
        cout << "Account Type: Savings Account\n";
        cout << "Interest Rate: " << interestRate << "%\n";
    }
};

class CurrentAccount : public Account {
private:
    double minBalance;
    double penalty;

public:
    CurrentAccount(string n = "", long acc = 0, double bal = 0.0,
                   double minBal = 1000.0, double pen = 50.0)
        : Account(n, acc, bal), minBalance(minBal), penalty(pen) {}

    // Override calculateInterest (no interest for current account)
    double calculateInterest() override {
        cout << "\n--- Current Account Interest ---\n";
        cout << "No interest for Current Account\n";

        // Check for penalty if below minimum balance
        if (balance < minBalance) {
            cout << "Balance below minimum ($" << minBalance << ")\n";
            cout << "Penalty charged: $" << penalty << endl;
            balance -= penalty;
            return -penalty;
        }
        return 0.0;
    }

    void displayDetails() {
        Account::displayDetails();
        cout << "Account Type: Current Account\n";
        cout << "Minimum Balance: $" << minBalance << endl;
        cout << "Penalty (if below min): $" << penalty << endl;
    }
};

int main() {

```

```

cout << "=== Banking System ===\n\n";

// Pointer to Object - dynamically create account objects
SavingsAccount* savings = new SavingsAccount("Alice Johnson", 1001,
5000.0, 5.0);
CurrentAccount* current = new CurrentAccount("Bob Smith", 2001, 800.0,
1000.0, 50.0);

cout << "--- Initial Account Details ---\n";
savings->displayDetails();
current->displayDetails();

// Pointer to Function - storing function addresses
cout << "\n\n=== Pointer to Function Demo ===\n";
void (Account::*depositFunc)(double) = &Account::deposit;
void (Account::*withdrawFunc)(double) = &Account::withdraw;
void (Account::*displayBalFunc)() = &Account::displayBalance;

cout << "Calling deposit through function pointer on Savings
Account:\n";
(savings->*depositFunc)(1000.0);

cout << "\nCalling withdraw through function pointer on Savings
Account:\n";
(savings->*withdrawFunc)(500.0);

cout << "\nCalling displayBalance through function pointer:\n";
(savings->*displayBalFunc)();

// Pointer to Data Member - accessing and modifying balance
cout << "\n\n=== Pointer to Data Member Demo ===\n";
double Account::*balancePtr = Account::getBalancePointer();

cout << "Accessing balance through pointer to data member:\n";
cout << "Savings Account balance: $" << savings->*balancePtr << endl;
cout << "Current Account balance: $" << current->*balancePtr << endl;

cout << "\nModifying balance through pointer to data member:\n";
savings->*balancePtr += 500.0;

```



```

    cout << "Added $500 to savings. New balance: $" <<
savings->*balancePtr << endl;

    // this Pointer demonstration
    cout << "\n\n=== this Pointer Demo ===\n";
    Account* thisPtr = savings->getThis();
    cout << "Address returned by getThis(): " << thisPtr << endl;
    cout << "Address of savings object: " << savings << endl;
    cout << "Addresses match: " << (thisPtr == savings ? "Yes" : "No") <<
endl;

    // Runtime Polymorphism - calling through base class pointer
    cout << "\n\n=== Runtime Polymorphism Demo ===\n";
    cout << "Calling calculateInterest() through base class pointer:\n";

    Account* accounts[2];
    accounts[0] = savings;
    accounts[1] = current;

    for (int i = 0; i < 2; i++) {
        cout << "\nAccount " << i + 1 << ":\n";
        double interest = accounts[i]->calculateInterest();
        accounts[i]->displayBalance();
    }

    // Virtual Function - demonstrating polymorphic behavior
    cout << "\n\n=== Virtual Function Demonstration ===\n";
    Account* ptr;

    ptr = savings;
    cout << "Base pointer -> Savings Account:\n";
    ptr->calculateInterest(); // Calls SavingsAccount version

    ptr = current;
    cout << "\nBase pointer -> Current Account:\n";
    ptr->calculateInterest(); // Calls CurrentAccount version

    // Operations menu
    cout << "\n\n=== Banking Operations ===\n";
    int choice;

```

```

do {
    cout << "\n1. Deposit to Savings\n";
    cout << "2. Withdraw from Savings\n";
    cout << "3. Display Savings Balance\n";
    cout << "4. Apply Interest to Savings\n";
    cout << "5. Display Current Account Details\n";
    cout << "6. Exit\n";
    cout << "Enter choice: ";
    cin >> choice;

    double amount;
    switch (choice) {
        case 1:
            cout << "Enter amount to deposit: $";
            cin >> amount;
            (savings->*depositFunc) (amount);
            break;
        case 2:
            cout << "Enter amount to withdraw: $";
            cin >> amount;
            (savings->*withdrawFunc) (amount);
            break;
        case 3:
            (savings->*displayBalFunc) ();
            break;
        case 4:
            savings->applyInterest();
            break;
        case 5:
            current->displayDetails();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
    }
} while (choice != 6);

// Final account details

```

```
cout << "\n\n=== Final Account Details ===\n";
savings->displayDetails();
current->displayDetails();

// Cleanup
delete savings;
delete current;

cout << "\n\nThank you for using our Banking System!\n";

return 0;
}
```